[NOME DA INSTITUIÇÃO DE ENSINO]
[CURSO] · [DISCIPLINA]

Vinicius Bueno
[DEMAIS INTEGRANTES DO GRUPO]

BARBTECH: MVP SEGURO DE AGENDAMENTO PARA BARBEARIAS

Relatório técnico de segurança da informação (Security by Design)

[CIDADE]
2026

Vinicius Bueno
[DEMAIS INTEGRANTES DO GRUPO]

BARBTECH: MVP SEGURO DE AGENDAMENTO PARA BARBEARIAS

Relatório técnico de segurança da informação (Security by Design)

Relatório técnico apresentado como projeto em grupo [AC2] da disciplina [NOME DA DISCIPLINA], do curso de [CURSO], da [INSTITUIÇÃO], como requisito parcial de avaliação.

Orientador(a): [NOME DO(A) PROFESSOR(A)]

[CIDADE]
2026

RESUMO

Este relatório documenta o projeto de segurança do BarbTech, um aplicativo móvel de agendamento entre clientes e profissionais de barbearia, desenvolvido com HTML e JavaScript empacotados pelo Capacitor para Android e iOS e apoiado em Firebase Authentication e Cloud Firestore, com dois serviços Node opcionais (assistente de inteligência artificial e serviço de segurança). O trabalho segue a abordagem de segurança desde o projeto (Security by Design) e um ciclo de desenvolvimento seguro, tomando como referência o OWASP Top 10, o ASVS, o MASVS, o SAMM e o NIST. São apresentados o escopo e os tipos de dados tratados, a arquitetura em camadas e a modelagem de ameaças pelo método STRIDE, ambas desenhadas no draw.io. O texto responde às quatro perguntas orientadoras (autenticação, proteção de dados, prevenção de ataques e riscos) e analisa onze riscos com impacto, probabilidade, pilares afetados, categoria STRIDE, artigos da LGPD, pontuação CVSS 3.1 e fraqueza CWE. A ordem de adequações proposta parte da pontuação de risco e do contexto do aplicativo. As evidências vêm de testes e comandos executados em ambiente simulado, e o relatório aponta com franqueza o que não foi testado e o que ainda é pendência, como a ausência de autenticação de segundo fator e a necessidade de atualizar dependências com vulnerabilidades conhecidas.

Palavras-chave: segurança da informação; modelagem de ameaças; LGPD; Firebase; aplicativo móvel.

ABSTRACT

This report documents the security design of BarbTech, a mobile scheduling app connecting customers and barbershop professionals, built with HTML and JavaScript packaged by Capacitor and backed by Firebase Authentication and Cloud Firestore, plus two optional Node services. Following Security by Design and a secure development lifecycle, it uses OWASP Top 10, ASVS, MASVS, SAMM and NIST as references. It presents scope, data classification, a layered architecture and a STRIDE threat model drawn in draw.io, answers the four guiding questions, and analyses eleven risks with impact, probability, affected pillars, STRIDE category, LGPD articles, CVSS 3.1 score and CWE. The remediation order is derived from the risk scores and the application context. Evidence comes from tests and commands run in a simulated environment, and the report states plainly what was not tested and what remains pending, such as the lack of a second authentication factor and dependencies with known vulnerabilities.

Keywords: information security; threat modeling; LGPD; Firebase; mobile application.

SUMÁRIO

1 INTRODUÇÃO 7

2 OBJETIVO E ESCOPO (SECURITY BY DESIGN) 7

2.1 Descrição da aplicação 7

2.2 Objetivo e problema real 8

2.3 Público-alvo 8

2.4 Tipos de dados e grau de sensibilidade 8

2.5 Ciclo de desenvolvimento seguro e escolhas técnicas 9

3 ARQUITETURA DO SISTEMA 10

3.1 Componentes e camadas 10

3.2 Fluxos e pontos de entrada de dados 12

4 MODELAGEM DE AMEAÇAS 12

5 PERGUNTAS ORIENTADORAS 14

5.1 Como será a autenticação? 14

5.2 Como proteger os dados? 15

5.3 Como evitar ataques? 16

5.4 Quais os riscos da aplicação? 17

6 ANÁLISE DE RISCOS E MATRIZ DE PRIORIZAÇÃO 18

6.1 Método 18

6.2 R1 – Força bruta e roubo de credenciais no login 18

6.3 R2 – Falha no controle de acesso do banco (regras do Firestore) 19

6.4 R3 – Exposição de dados pessoais e de saúde 19

6.5 R4 – Tomada de conta pelo código de recuperação 20

6.6 R5 – Injeção de script ou conteúdo na tela (XSS) 20

6.7 R6 – Abuso do backend do assistente de IA 21

6.8 R7 – Dados calculados ou decididos no aparelho (adulteração) 21

6.9 R8 – Dependências com vulnerabilidades e cadeia de suprimentos 22

6.10 R9 – Aparelho perdido, roubado ou emprestado 22

6.11 R10 – Indisponibilidade e esgotamento de recursos 23
6.12 R11 – Governança e conformidade com a LGPD 23
6.13 Ordem de priorização das adequações 24
7 VERIFICAÇÃO E EVIDÊNCIAS 25
7.1 Telas do aplicativo 25
7.2 Testes automatizados e comandos 27
8 CONSIDERAÇÕES FINAIS 31

1 INTRODUÇÃO

Aplicativos de agendamento reúnem, em um só lugar, dados que costumam ser tratados com pouco cuidado: nome, telefone, endereço, CPF e, no caso do BarbTech, até uma observação de saúde (como alergia a produtos) que o cliente pode informar para o atendimento em casa. Um vazamento ou uma adulteração desses dados atinge pessoas reais e expõe os responsáveis pelo sistema à Lei Geral de Proteção de Dados Pessoais (BRASIL, 2018). Por isso, a segurança foi tratada como requisito de arquitetura, e não como correção posterior.

Este relatório descreve o BarbTech sob essa ótica. Ele apresenta o escopo e os dados tratados, a arquitetura em camadas, a modelagem de ameaças, as respostas às perguntas orientadoras sobre autenticação, proteção de dados, prevenção de ataques e riscos, a análise de onze riscos com a ordem de adequações, e as evidências dos testes executados. Todas as afirmações sobre o sistema se referem ao código do projeto na sua versão atual. Quando algo não foi testado ou ainda não existe, o texto diz isso de forma explícita.

A metodologia combina quatro referências. O OWASP Top 10 de 2021 orientou a escolha das categorias de risco; o Application Security Verification Standard (ASVS) e o Mobile Application Security Verification Standard (MASVS) orientaram os controles de aplicação e de aplicativo móvel; o modelo de maturidade SAMM orientou o plano de evolução; e o Cybersecurity Framework do NIST deu a estrutura de identificar, proteger, detectar, responder e recuperar (OWASP, 2021a; 2021b; 2023; 2020; NIST, 2024). A modelagem de ameaças seguiu o método STRIDE (SHOSTACK, 2014), aplicado manualmente sobre o diagrama de arquitetura desenhado no draw.io. As ferramentas OWASP Threat Dragon, Microsoft Threat Modeling Tool e Devici não foram utilizadas neste trabalho.

2 OBJETIVO E ESCOPO (SECURITY BY DESIGN)

2.1 Descrição da aplicação

O BarbTech é um aplicativo de agendamento para barbearias. O cliente cria uma conta, informa seu CEP, encontra profissionais da sua região, escolhe um serviço, vê os horários livres e marca o atendimento na barbearia ou em casa. O profissional cadastra seus serviços, define os dias e horários de trabalho, acompanha a agenda, conclui atendimentos e acompanha resultados. Um assistente virtual, opcional, conversa com o cliente em linguagem natural e o ajuda a escolher horário. O aplicativo é uma página HTML com JavaScript executada dentro do WebView do Capacitor (Android e iOS) e conversa diretamente com o Firebase Authentication e o Cloud Firestore, sem um servidor de aplicação próprio no caminho principal.

Existem ainda três componentes de servidor. O primeiro é uma Cloud Function, que envia avisos no celular quando um agendamento muda. O segundo é o backend do assistente, em Node com Express, que recebe a conversa e a repassa ao provedor de inteligência artificial. O terceiro é um serviço de segurança, também em Node, opcional, que oferece envio de código por e-mail, bloqueio de login entre aparelhos e trilha de auditoria. O serviço de segurança deixou de ser necessário para a recuperação de conta, que hoje funciona sem servidor próprio.

2.2 Objetivo e problema real

O problema real é a desorganização do agendamento em barbearias pequenas e de profissionais autônomos: horários combinados por mensagem, conflitos de agenda, faltas sem aviso e nenhuma visão de resultados. O BarbTech resolve isso com reserva de horário única (dois clientes não conseguem ocupar o mesmo horário), avisos ao profissional e ao cliente e um painel de resultados. O objetivo de segurança é permitir que isso aconteça tratando dados pessoais com o mínimo necessário, garantindo que cada pessoa acesse apenas o que é seu e que o sistema continue disponível e auditável.

2.3 Público-alvo

O público tem dois perfis. Os clientes são adultos que procuram barbearia na sua região, usam o aplicativo pelo celular e não têm conhecimento técnico de segurança. Os profissionais são barbeiros autônomos ou donos de barbearia que cadastram CPF ou CNPJ e endereço. O aplicativo é destinado a maiores de 18 anos, o que afasta, por decisão de escopo, as exigências adicionais para dados de crianças e adolescentes do artigo 14 da LGPD. Como o público não é técnico, os controles precisam proteger por padrão, sem depender de configuração feita pelo usuário.

2.4 Tipos de dados e grau de sensibilidade

Os dados foram classificados em quatro níveis. O nível público inclui o nome do profissional, a foto, o banner, os serviços e a posição aproximada: tudo isso é mostrado aos clientes por finalidade do próprio serviço. A posição do profissional é arredondada para cerca de um quilômetro, de modo que o endereço exato nunca fica público. O nível pessoal comum inclui e-mail, telefone, estado, cidade, CEP e os agendamentos, que só o titular e a outra parte do agendamento enxergam. O nível pessoal de alto risco inclui CPF, CNPJ e endereço residencial, guardados em uma área privada, que só o dono lê, e usados apenas para o cadastro. O nível sensível, conforme o artigo 5º, inciso II, e o artigo 11 da LGPD, é a condição de saúde que o cliente pode descrever (por exemplo, alergia), compartilhada com o profissional somente nos atendimentos em casa e somente se o cliente consentir.

Há ainda os dados de segurança, que não pertencem ao titular, mas protegem a sua conta: o token de sessão emitido pelo Firebase, o cofre do código de recuperação (a senha criptografada com uma chave derivada do código), os registros de tentativas de login e o registro de consentimento, que guarda a versão da política aceita e a data do aceite.

2.5 Ciclo de desenvolvimento seguro e escolhas técnicas

O desenvolvimento seguiu as fases de um ciclo de desenvolvimento seguro (SSDLC) adaptadas a um grupo pequeno. Na fase de requisitos, os requisitos de segurança foram escritos junto com os funcionais, a partir do ASVS: autenticação (capítulo 2), sessão (capítulo 3), controle de acesso (capítulo 4), validação de entrada (capítulo 5), criptografia (capítulo 6), tratamento de erros e registros (capítulo 7), proteção de dados (capítulo 8), comunicação (capítulo 9) e configuração (capítulo 14). Na fase de projeto, a arquitetura foi desenhada e submetida à modelagem de ameaças (seções 3 e 4). Na implementação, adotaram-se bibliotecas consolidadas em vez de código próprio para o que é crítico, e a única criptografia escrita pelo grupo usa as funções nativas do navegador e do Node. Na verificação, foram feitos testes automatizados, testes ponta a ponta com o Firebase simulado, varredura de segredos e auditoria de dependências (seção 7). Na liberação e na operação, previu-se o plano de adequações da seção 6.13.

As escolhas técnicas mais importantes foram cinco. A primeira foi delegar a autenticação ao Firebase Authentication, em vez de guardar senhas em um banco próprio, porque a proteção de senhas e o controle de abuso são problemas resolvidos por quem opera a plataforma em escala. A segunda foi concentrar a autorização nas regras de segurança do Firestore, avaliadas pelo servidor do Google a cada leitura e escrita, porque tudo o que roda no aparelho pode ser alterado por quem o controla. A terceira foi separar os dados em coleções por finalidade e por dono, com uma área privada para CPF e endereço. A quarta foi manter chaves de terceiros apenas em servidor (arquivo de ambiente do backend), nunca no aplicativo. A quinta foi tornar opcionais os servidores próprios, para que o menor caminho de produção tenha poucos componentes a proteger.

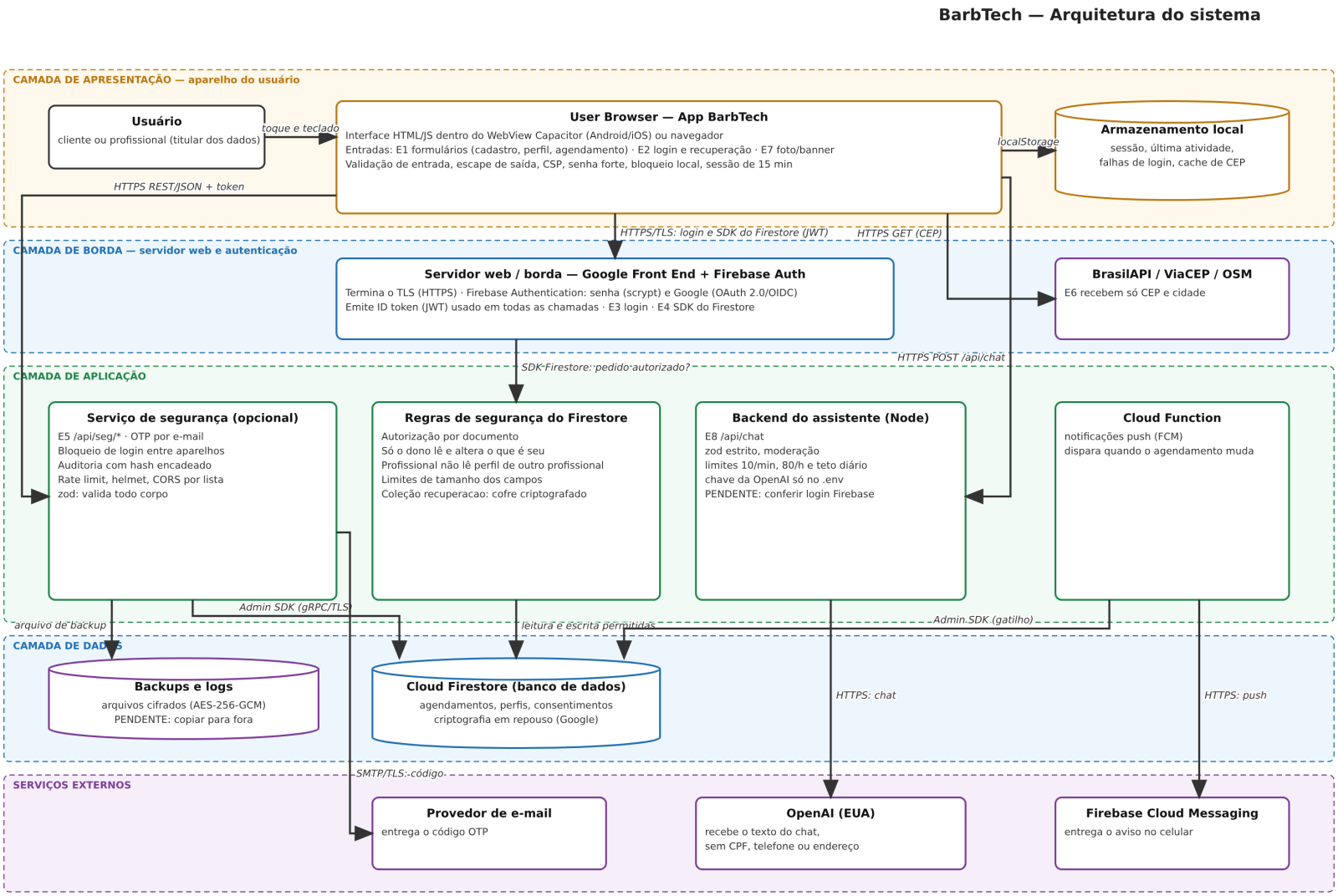
3 ARQUITETURA DO SISTEMA

3.1 Componentes e camadas

A Figura 1 mostra a arquitetura em camadas. A camada de apresentação é o aparelho do usuário, onde roda o aplicativo (o *User Browser* do modelo clássico, aqui um WebView do Capacitor, ou um navegador comum quando o aplicativo é aberto como página). A camada de borda cumpre o papel do servidor web: é a infraestrutura do Google (Google Front End), que encerra a conexão segura (TLS) e dá acesso ao Firebase Authentication. A camada de aplicação reúne as regras de segurança do Firestore, que decidem se cada pedido é permitido, e os serviços em Node (assistente e segurança) e a Cloud Function. A camada de dados é o Cloud Firestore, acompanhado dos backups do serviço de segurança. Serviços externos (provedor de inteligência artificial, e-mail, mensagens push e consulta de CEP) aparecem em uma camada própria, por serem de terceiros.

Em relação ao modelo pedido na atividade (navegador, servidor web como o Apache, aplicação como o PHP e banco de dados como o MySQL), a correspondência é a seguinte. O navegador é o aplicativo. O servidor web é a borda do Google com o Firebase Authentication. A aplicação é o conjunto formado pelas regras do Firestore e pelos serviços Node. O banco de dados é o Cloud Firestore, que é um banco de documentos e, por isso, não usa SQL: o fluxo que, no modelo clássico, seria SQL aparece como chamadas do SDK do Firestore por TLS.

Figura 1 – Arquitetura do BarbTech em camadas, com fluxos nomeados e pontos de entrada E1 a E8



Fonte: elaborado pelo autor (2026), no draw.io. O arquivo editável é BarbTech-Arquitetura-ThreatModel.drawio.

3.2 Fluxos e pontos de entrada de dados

Os fluxos entre as camadas são todos protegidos por TLS e estão nomeados na Figura 1. O aplicativo fala com a borda por HTTPS para fazer login e, em seguida, usa o SDK do Firestore, que carrega o token de sessão (um JWT) em cada pedido. A borda encaminha o pedido às regras do Firestore, que o autorizam ou negam, e só então o banco é lido ou escrito. O aplicativo chama diretamente, também por HTTPS, o backend do assistente e o serviço de segurança, e consulta os serviços públicos de CEP.

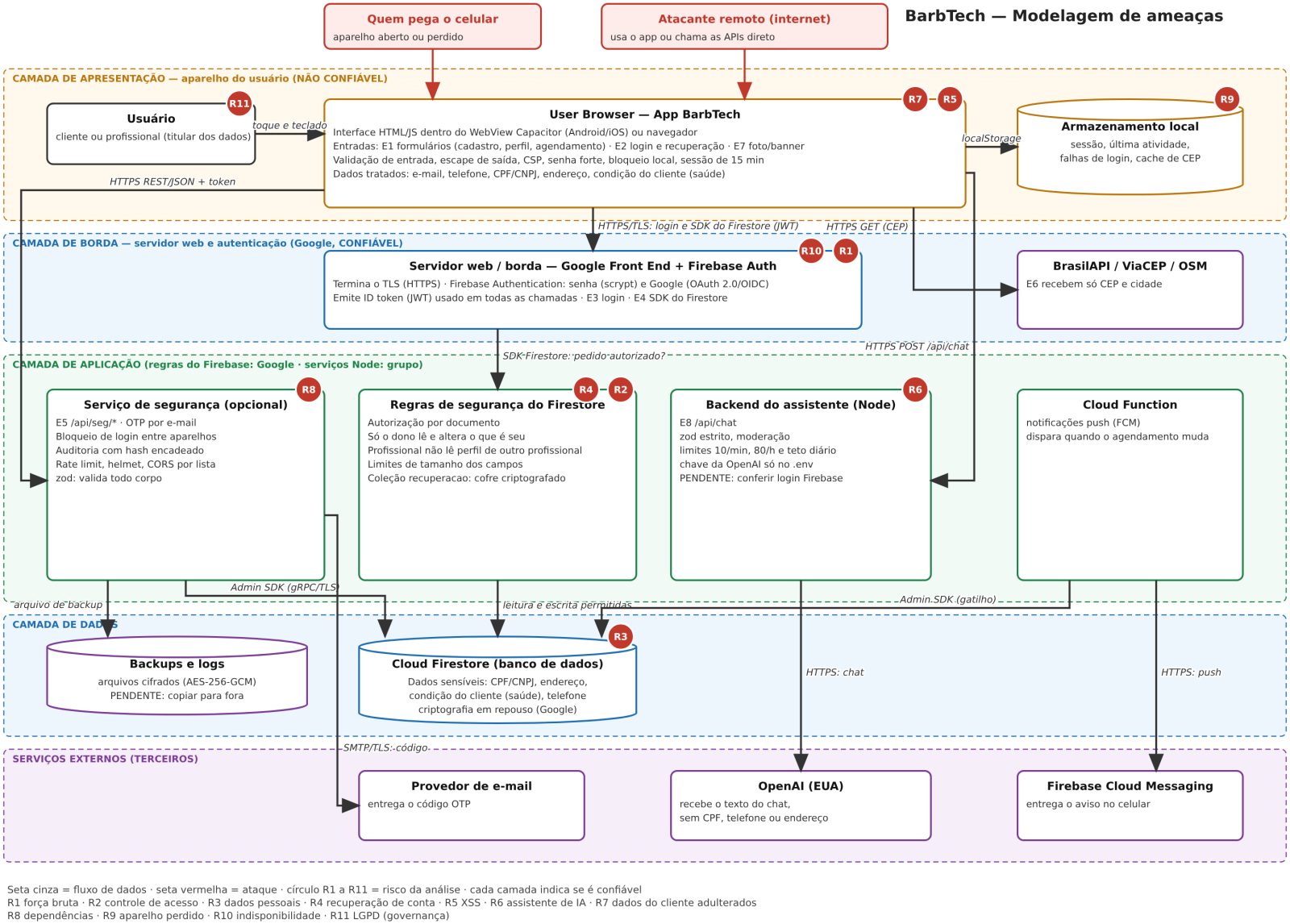
Os pontos de entrada de dados são oito. O E1 são os formulários do aplicativo (cadastro, perfil, agendamento), onde o usuário digita texto livre. O E2 é o login e a recuperação de conta. O E3 é o login pelo Firebase Authentication, inclusive o login com Google (OAuth 2.0 e OpenID Connect). O E4 é o SDK do Firestore, por onde chegam todas as leituras e escritas. O E5 são os endereços */api/seg* do serviço de segurança. O E6 são as consultas de CEP. O E7 é o envio de foto e banner, que passa por redimensionamento no aparelho. O E8 é o endereço */api/chat* do assistente. Os pontos E5 e E8 são os únicos que expõem um servidor próprio à internet, e, por isso, recebem validação rigorosa, limite de requisições e política de origem (CORS).

4 MODELAGEM DE AMEAÇAS

A Figura 2 atualiza o diagrama de arquitetura com os pontos de entrada, os dados sensíveis mapeados, as fronteiras de confiança e os onze riscos analisados na seção 6, marcados de R1 a R11. A modelagem foi feita no próprio draw.io, de forma manual, aplicando o STRIDE a cada elemento e a cada fluxo: para cada componente perguntou-se como ele poderia ser falsificado, adulterado, negado, ter informação divulgada, ficar indisponível ou ter privilégio elevado (SHOSTACK, 2014).

O diagrama define três fronteiras de confiança. A primeira é o aparelho do usuário, que não é confiável: quem controla o aparelho pode alterar o código do aplicativo, ler o armazenamento local e chamar as interfaces diretamente, e por isso nenhuma regra de segurança que dependa apenas do aplicativo é considerada garantia. A segunda é a infraestrutura do Google, considerada confiável e gerenciada, onde são aplicadas as regras do Firestore e a autenticação. A terceira são os servidores próprios do grupo e os serviços de terceiros, que exigem controles próprios. Os atacantes considerados são duas pessoas: a que pega um celular aberto ou perdido e o atacante remoto, que pode usar o aplicativo normalmente ou ignorá-lo e chamar as interfaces do Firebase e dos servidores diretamente.

Figura 2 – Modelagem de ameaças do BarbTech: fronteiras de confiança, atacantes, dados sensíveis e riscos R1 a R11



Fonte: elaborado pelo autor (2026), no draw.io. O arquivo editável é BarbTech-Arquitetura-ThreatModel.drawio.

Os dados sensíveis estão concentrados em dois lugares: o banco (CPF, CNPJ, endereço e condição de saúde) e o formulário do aplicativo, onde são digitados. Os riscos mais expostos à internet são os da borda de login (R1 e R10), o do assistente (R6) e o das regras do banco (R2). O risco R9, do aparelho perdido, e os riscos R5 e R7, de código e dados controlados pelo cliente, ficam na camada não confiável.

5 PERGUNTAS ORIENTADORAS

5.1 Como será a autenticação?

Protocolos. A autenticação com e-mail e senha é feita pelo Firebase Authentication. Após o login, o serviço emite um token de identidade no formato JSON Web Token (IETF, 2015), assinado pelo Google e válido por uma hora, renovado por um token de atualização. Esse token acompanha cada pedido ao Firestore e é o que as regras usam para saber quem está pedindo. O login com Google usa o fluxo OAuth 2.0 (IETF, 2012) com OpenID Connect: o aplicativo recebe o token do Google e o troca por uma sessão do Firebase. O protocolo SAML não é usado, porque não há provedor de identidade corporativo no escopo.

Senha. O Firebase guarda as senhas com uma função de derivação própria da família scrypt, de modo que o grupo nunca tem acesso às senhas nem às suas versões embaralhadas. O aplicativo exige, no cadastro, na recuperação e na redefinição, uma senha de pelo menos oito caracteres, com letra maiúscula, letra minúscula, número e caractere especial, com limite de 128 caracteres, e mostra ao usuário uma barra de força e a lista de requisitos atendidos. Essa política é validada no aparelho, para orientar quem digita; a garantia do lado do servidor é o requisito mínimo do próprio Firebase, que é de seis caracteres. Isso é uma limitação: quem chamar a interface do Firebase diretamente consegue criar conta com uma senha mais fraca. A recomendação do NIST (2017) de verificar a senha contra listas de senhas vazadas não foi implementada.

Autenticação multifator. O aplicativo não implementa autenticação de segundo fator para contas com senha. Uma versão anterior do projeto tinha um segundo fator por aplicativo autenticador (TOTP, conforme a RFC 6238), executado pelo serviço de segurança, mas o grupo decidiu retirá-lo para não depender de um servidor próprio. Em seu lugar existem controles que reduzem o risco, mas que não são um segundo fator: política de senha forte, bloqueio de tentativas e o código de recuperação. Quem entra pelo Google herda a verificação em duas etapas que a própria conta Google tiver ativada. A adoção do segundo fator pelo Firebase exige, segundo a documentação do serviço, a atualização para o Identity Platform, o que ainda precisa ser confirmado e orçado. Por essa razão o risco R1 permanece classificado como alto (seção 6.2).

Gestão de acessos e sessões. O acesso é separado por papel. Uma conta é de cliente ou de profissional, e a existência do documento do perfil em uma das coleções decide o papel; as

regras do Firestore usam essa mesma informação. Cada documento tem um dono, e só o dono lê e altera seus dados, com exceções explícitas: o cliente enxerga os perfis públicos e as agendas dos profissionais, e o profissional enxerga os agendamentos em que é parte. Um profissional não consegue ler o perfil de outro profissional. A sessão pode ser mantida no aparelho ou apenas na janela do aplicativo, conforme a escolha do usuário no login. O aplicativo encerra a sessão depois de 15 minutos sem uso, tanto com o aplicativo aberto quanto ao reabri-lo. Esse prazo é aplicado no aparelho; o token do Firebase continua válido por até uma hora, e revogá-lo antes disso exige o serviço de segurança.

Recuperação de conta. A pessoa recebe, no cadastro, um código único de 20 caracteres, exibido uma única vez em uma janela que permite baixar um arquivo ou copiar o código, e só continua depois de confirmar que o guardou. Quem esquece a senha informa o e-mail, o código e a nova senha. O mecanismo funciona sem servidor próprio: a senha da conta é criptografada no aparelho, com AES-256-GCM, usando uma chave derivada do código por PBKDF2 com SHA-256 e 600 mil iterações, e o resultado é guardado em uma coleção do Firestore cujo identificador é o e-mail. Sem o código, o conteúdo não pode ser aberto, e o código tem 100 bits de aleatoriedade, o que torna inviável adivinhá-lo. Depois de usar o código, o sistema troca a senha, cria um código novo e invalida o antigo. As limitações desse desenho estão na seção 6.5. Existe, como alternativa, o envio de um link de redefinição pelo Firebase.

5.2 Como proteger os dados?

Em trânsito. Toda a comunicação com o Firebase, os servidores próprios e as APIs de CEP usa HTTPS. O Android foi configurado para recusar tráfego sem criptografia (*usesCleartextTraffic* desativado). Os certificados e a versão do TLS são os do Google para o Firebase; para os servidores próprios, o certificado depende de onde forem hospedados, o que não faz parte do pacote entregue.

Em repouso. O Cloud Firestore criptografa os dados em repouso por padrão (GOOGLE CLOUD, 2024). Esse é o único nível de criptografia aplicado ao CPF, ao endereço e à condição de saúde: não há criptografia de campo com chave controlada pelo grupo, e quem tiver acesso ao banco pelo console do Firebase lê esses valores. No aparelho, a sessão e as falhas de login ficam no armazenamento local do WebView, e o Android foi configurado para não incluir os dados do aplicativo em cópias de segurança (*allowBackup* desativado). O cofre de recuperação e os arquivos de backup do serviço de segurança são criptografados com AES-256-GCM.

Classificação. A classificação em quatro níveis (público, pessoal comum, pessoal de alto risco e sensível) está descrita na seção 2.4 e se reflete na estrutura do banco: o perfil público fica em uma coleção que os clientes leem; CPF, CNPJ e endereço do profissional ficam em uma subcoleção privada, só do dono; a condição de saúde do cliente fica no perfil do

cliente, que só ele lê, e só é repassada ao profissional no atendimento em casa e com consentimento.

Mascaramento e anonimização. Há mascaramento por redução de precisão: a posição de um profissional é arredondada para cerca de um quilômetro antes de ser publicada. O texto enviado ao assistente é limpo de caracteres especiais e não inclui CPF, telefone ou endereço. No serviço de segurança, o e-mail registrado na trilha de auditoria é substituído por um código gerado com HMAC. Não há anonimização de dados para estatística: os resultados mostrados ao profissional vêm de seus próprios agendamentos. Na exportação de dados do titular (artigo 18 da LGPD), o arquivo traz tudo o que o aplicativo guarda sobre ele, com a foto substituída por um marcador.

5.3 Como evitar ataques?

Firewall, proxy e WAF. O caminho principal (aplicativo, Firebase) fica atrás da infraestrutura do Google, que já oferece absorção de ataques de negação de serviço e filtragem de rede. O grupo não opera firewall, proxy reverso nem WAF próprios. Os servidores Node, quando forem publicados, devem ficar atrás de um proxy reverso com TLS e, de preferência, de um WAF do provedor; isso é recomendação, não algo entregue.

Limitação de requisições. Os dois servidores Node usam limitação por IP (express-rate-limit). O assistente aceita 10 requisições por minuto e 80 por hora por origem, além de um teto diário de requisições para controlar custo. O serviço de segurança aplica 120 requisições por minuto por IP e limites mais rigorosos para login, envio de código e recuperação, além de contar tentativas por e-mail. A Figura 9 mostra o efeito: de 130 requisições seguidas, 12 receberam resposta 429. No Firebase Authentication, o limite contra tentativas em excesso é do próprio serviço. O bloqueio do aplicativo após cinco senhas erradas (5, 15 e 60 minutos) é local e pode ser contornado por quem reinstalar o aplicativo ou chamar o Firebase diretamente.

Validação e sanitização de entrada. No aplicativo, todo texto passa por uma função que remove caracteres de controle e os sinais de menor e maior, normaliza espaços e limita o tamanho. Os campos têm limite de caracteres e máscara (CPF, CNPJ e telefone), e CPF e CNPJ têm os dígitos verificadores conferidos (a Figura 3 mostra a recusa de um CPF inválido). As regras do Firestore repetem os limites de tamanho, mas não conferem os dígitos. Nos servidores, a biblioteca zod valida o formato, o tamanho e os tipos de cada corpo, e corpos acima de 8 kilobytes (segurança) ou 64 kilobytes (assistente) são recusados. Como o Firestore não usa SQL, não há injeção de SQL; o equivalente seria a injeção de operadores em consultas, evitada porque o aplicativo nunca monta consultas com texto do usuário. Na saída, todo conteúdo exibido passa por uma função de escape antes de ir para a página.

Cabeçalhos de segurança. Os servidores Node usam a biblioteca helmet, e a Figura 9 mostra o resultado: política de segurança de conteúdo, HSTS, *nosniff*, *X-Frame-Options* e

política de referência. A origem dos pedidos é restrita por lista (a Figura 9 mostra a resposta 403 a uma origem não autorizada), e erros devolvem apenas uma mensagem genérica. A página do aplicativo declara uma política de segurança de conteúdo por meio de uma etiqueta *meta*, que restringe origens de rede e de scripts, mas ainda permite scripts embutidos (*unsafe-inline*), o que enfraquece a proteção contra XSS e é uma pendência.

Dependências. O projeto usa arquivos de travamento (*package-lock.json*) e foi submetido ao *npm audit* na entrega deste relatório. O resultado, na Figura 11, é um achado real: 11 vulnerabilidades no serviço de segurança (4 de severidade alta) e 2 no assistente (ambas altas), todas com correção apenas por atualização de versão principal. O risco e o plano estão na seção 6.9. O aplicativo também carrega os módulos do Firebase diretamente do domínio do Google, sem verificação de integridade, o que é uma dependência de cadeia de suprimentos aceita nesta versão.

5.4 Quais os riscos da aplicação?

Os riscos específicos do contexto são onze e estão detalhados na seção 6. Em resumo: tentativas de adivinhar senhas sem segundo fator (R1); regras de acesso do banco que, se falharem, expõem todos os dados (R2); exposição de dados pessoais e de saúde (R3); tomada de conta pela recuperação (R4); injeção de código na tela (R5); abuso do assistente de IA, que hoje não confere quem é o usuário (R6); adulteração de dados calculados no aparelho, como o preço (R7); dependências com vulnerabilidades (R8); aparelho perdido (R9); indisponibilidade (R10); e governança da LGPD (R11). O impacto real é maior para os dados do cliente, por incluírem CPF, endereço e uma informação de saúde, e para a confiança dos profissionais, cuja renda depende da agenda.

Quanto à ordem das ações de conformidade, o plano está na seção 6.13. Quanto à cultura de segurança e às boas práticas administrativas, o grupo adota cinco práticas: revisão de código por outro integrante antes de entrar na versão principal, com uma lista de conferência de segurança (autorização, validação, segredos, registros); proibição de segredos no repositório, com varredura antes de cada entrega (Figura 10); execução do *npm audit* e dos testes a cada alteração; revisão trimestral deste relatório e do modelo de ameaças, ou a cada funcionalidade nova que trate dados; e treinamento básico de todos os integrantes sobre o OWASP Top 10 e sobre os princípios e direitos da LGPD. Para os profissionais, o aplicativo deve explicar de forma simples por que não se deve compartilhar o código de recuperação nem a senha. Como o grupo não tem um encarregado de dados definido, a designação de um responsável (LGPD, artigo 41) e de um procedimento de resposta a incidentes, com comunicação à autoridade e aos titulares nos termos do artigo 48, é a primeira ação administrativa.

6 ANÁLISE DE RISCOS E MATRIZ DE PRIORIZAÇÃO

6.1 Método

Cada risco foi avaliado por sete parâmetros. O primeiro é a descrição completa. O segundo é o cálculo do risco real: o impacto, de 1 (insignificante) a 5 (severo), multiplicado pela probabilidade, de 1 (raro) a 5 (quase certo), resultando em um valor de 1 a 25. O risco real já considera os controles hoje implementados, e classifica-se como baixo até 4, médio de 5 a 9, alto de 10 a 15 e crítico a partir de 16. O terceiro parâmetro são os pilares da segurança da informação afetados. Adotaram-se oito pilares: confidencialidade, integridade, disponibilidade, autenticidade, irretratabilidade (não repúdio), legalidade (conformidade com leis), privacidade e auditabilidade. O quarto é a associação com o modelo STRIDE, o quinto é o artigo da LGPD afetado, e o sexto e o sétimo são a fraqueza catalogada (CWE) e a pontuação de severidade (CVSS).

As pontuações CVSS seguem a especificação 3.1 (FIRST, 2019), com o vetor registrado em cada risco. Os vetores são estimativas do grupo para o pior caso plausível de cada fraqueza, e não pontuações oficiais de um cadastro de vulnerabilidades; o cálculo foi feito por um programa conferido com vetores de valor conhecido. Os identificadores de fraqueza são os do catálogo CWE (MITRE, 2024). O risco R11 é de governança, e não de uma falha técnica, e por isso não recebe pontuação CVSS.

6.2 R1 – Força bruta e roubo de credenciais no login

Um atacante testa senhas de forma automática contra contas conhecidas ou usa credenciais vazadas de outros serviços. Se acertar, lê agendamentos, telefone, endereço e a condição de saúde de um cliente, ou altera a agenda de um profissional. O dano é alto porque a conta concentra todos os dados do titular, e a probabilidade é moderada porque o aplicativo, sem segundo fator, depende de uma única senha.

Risco real. Impacto 4 multiplicado por probabilidade 3 resulta em 12, risco alto. **Pilares afetados:** confidencialidade, integridade, autenticidade e privacidade. **STRIDE:** falsificação de identidade (*Spoofing*). **LGPD:** artigo 46 e artigo 6º, inciso VII (segurança). **Vulnerabilidades associadas:** CWE-307 (restrição imprópria de tentativas de autenticação em excesso) e CWE-521 (requisitos fracos de senha); CVSS 3.1, vetor AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:N, pontuação 7,4.

Situação atual e tratamento. Há política de senha forte, controle de abuso do próprio Firebase, bloqueio local de cinco falhas com tempos de 5, 15 e 60 minutos, mensagem de erro idêntica para e-mail inexistente e senha errada e encerramento da sessão por inatividade. As limitações são três: a política é verificada só no aplicativo; o bloqueio local pode ser contornado; e não há segundo fator. A ação prevista é ativar a verificação em duas etapas,

orientar o uso do login com Google e comparar senhas novas com listas de senhas vazadas.

6.3 R2 – Falha no controle de acesso do banco (regras do Firestore)

Como o aplicativo fala direto com o banco, a única barreira entre um usuário e os dados de todos os outros são as regras de segurança. Uma regra errada, ou publicada em versão antiga, permitiria ler agendamentos alheios, alterar o preço ou o status de um atendimento ou ler perfis que deveriam ser privados. Um caso real aconteceu durante o desenvolvimento: profissionais conseguiam ler a lista de todos os outros profissionais, o que foi corrigido restringindo a leitura aos clientes e ao próprio dono. O impacto máximo é a exposição de toda a base, e a probabilidade é baixa apenas porque as regras foram revisadas, mas elas ainda não foram executadas no emulador do Firebase.

Risco real. Impacto 5 multiplicado por probabilidade 2 resulta em 10, risco alto. **Pilares afetados:** confidencialidade, integridade, privacidade e autenticidade. **STRIDE:** elevação de privilégio (*Elevation of Privilege*) e adulteração (*Tampering*). **LGPD:** artigo 46 e artigo 6º, inciso VII; artigo 11 quando há dado de saúde. **Vulnerabilidades associadas:** CWE-862 (autorização ausente) e CWE-639 (acesso por chave controlada pelo usuário, o IDOR); CVSS 3.1, vetor AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:L/A:N, pontuação 7,1.

Situação atual e tratamento. As regras negam por padrão e liberam cada coleção por dono: o cliente só lê o próprio perfil e o dos profissionais; o profissional só lê o próprio perfil e os agendamentos em que participa; cada lado altera apenas os campos que lhe cabem em um agendamento; um horário só pode ser reservado uma vez; os campos têm limite de tamanho. Pendências: executar os testes de regras no emulador, o que dá a evidência que falta, e publicar sempre o arquivo mais recente no console do Firebase.

6.4 R3 – Exposição de dados pessoais e de saúde

CPF, CNPJ, endereço e a observação de saúde do cliente são os dados de maior valor para um atacante e de maior dano para o titular, que pode sofrer fraude, assédio ou discriminação. A exposição pode ocorrer por falha de regra (R2), por conta tomada (R1, R4), por acesso indevido ao console do Firebase ou por cópia dos dados no aparelho. A probabilidade é baixa porque a área privada tem regra própria e o Android não permite cópia de segurança do aplicativo, mas os dados não têm criptografia de campo.

Risco real. Impacto 5 multiplicado por probabilidade 2 resulta em 10, risco alto. **Pilares afetados:** confidencialidade e privacidade. **STRIDE:** divulgação de informação (*Information Disclosure*). **LGPD:** artigo 5º, inciso II, artigos 11 e 46; artigo 6º, inciso III (necessidade). **Vulnerabilidades associadas:** CWE-200 (exposição de informação sensível) e CWE-312 (armazenamento de dado sensível sem criptografia de campo); CVSS 3.1, vetor AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N, pontuação 6,5.

Situação atual e tratamento. O perfil público não contém CPF nem endereço; a posição é arredondada; a condição de saúde só segue ao profissional no atendimento em casa e com consentimento, que o titular pode retirar; a criptografia em repouso do Google protege contra acesso físico ao armazenamento, mas não contra quem tem acesso ao console. A ação prevista é restringir quem acessa o console (contas com verificação em duas etapas e papéis mínimos), avaliar a criptografia de campo para CPF e minimizar o que se pede no cadastro.

6.5 R4 – Tomada de conta pelo código de recuperação

O código de recuperação é, na prática, uma segunda chave da conta. Quem o obtiver (por exemplo, de um arquivo baixado e esquecido em uma pasta compartilhada) entra na conta mesmo sem a senha. Além disso, o desenho sem servidor guarda no Firestore, em um documento cujo identificador é o e-mail, a senha criptografada pela chave do código. Esse documento pode ser lido por qualquer pessoa que saiba o e-mail, o que permite tentar abri-lo offline e revela se um e-mail tem cadastro. O ataque offline é inviável com um código de 100 bits e 600 mil iterações de PBKDF2, mas o custo de proteger o código passa a ser do usuário.

Risco real. Impacto 5 multiplicado por probabilidade 2 resulta em 10, risco alto. **Pilares afetados:** confidencialidade, autenticidade e privacidade. **STRIDE:** falsificação de identidade e divulgação de informação. **LGPD:** artigo 46 e artigo 6º, inciso VII. **Vulnerabilidades associadas:** CWE-640 (mecanismo fraco de recuperação de senha) e CWE-203 (diferença observável que revela a existência do e-mail); CVSS 3.1, vetor AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:N, pontuação 7,4.

Situação atual e tratamento. O código é mostrado uma vez e o usuário precisa confirmar que o guardou; ao ser usado, é trocado; a mensagem de erro é igual para e-mail sem cofre e código errado; as tentativas erradas são limitadas no aparelho; só o dono cria, troca ou apaga o próprio cofre. Limites: se a senha for trocada pelo link do Firebase, o código deixa de funcionar; o bloqueio de tentativas é só local; e perder o código e o acesso ao e-mail significa perder a conta. Como alternativa mais forte, a recuperação por link ou código enviado ao e-mail pelo servidor de segurança continua disponível quando o servidor é publicado.

6.6 R5 – Injeção de script ou conteúdo na tela (XSS)

Textos digitados por um usuário (nome, serviço, observações) podem ser exibidos a outro, como o nome do cliente na agenda do profissional. Se algum campo fosse inserido na página sem tratamento, um script malicioso poderia ler a sessão ou agir em nome da vítima. A probabilidade é baixa porque o aplicativo escapa toda saída e limpa a entrada, mas a política de segurança de conteúdo permite scripts embutidos, que reduzem a defesa em profundidade.

Risco real. Impacto 4 multiplicado por probabilidade 2 resulta em 8, risco médio. **Pilares afetados:** confidencialidade, integridade e autenticidade. **STRIDE:** adulteração e elevação de privilégio. **LGPD:** artigo 46. **Vulnerabilidades associadas:** CWE-79

(neutralização imprópria de entrada na geração de página); CVSS 3.1, vetor AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N, pontuação 6,1.

Situação atual e tratamento. Toda entrada passa por limpeza (sem sinais de menor e maior e sem caracteres de controle), todo texto exibido passa por escape, as imagens são recodificadas no aparelho e a política de segurança de conteúdo limita as origens de rede. A ação prevista é retirar os scripts embutidos da página e remover a permissão *unsafe-inline* da política.

6.7 R6 – Abuso do backend do assistente de IA

O endereço */api/chat* aceita pedidos sem comprovar quem os envia. Qualquer pessoa que descubra o endereço consegue consumir a cota paga do provedor de IA, o que é um risco de custo e de indisponibilidade, e pode tentar manipular o assistente com instruções maliciosas (injeção de *prompt*) para extrair os dados que o aplicativo envia como contexto, como a lista de agendamentos do cliente. O impacto é médio, e a probabilidade é alta porque o endereço é público e a verificação de login ainda não existe.

Risco real. Impacto 3 multiplicado por probabilidade 4 resulta em 12, risco alto. **Pilares afetados:** disponibilidade, confidencialidade e privacidade. **STRIDE:** negação de serviço e elevação de privilégio. **LGPD:** artigo 33 (transferência internacional, pois o provedor é estrangeiro), artigo 46 e artigo 6º, inciso III. **Vulnerabilidades associadas:** CWE-306 (autenticação ausente em função crítica) e CWE-770 (alocação de recursos sem limites); CVSS 3.1, vetor AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:L/A:L, pontuação 6,5.

Situação atual e tratamento. O servidor valida o corpo com esquema estrito, aplica moderação de conteúdo, limita a 10 pedidos por minuto, 80 por hora e um teto diário, restringe origens, não guarda a conversa no provedor e mantém a chave apenas no arquivo de ambiente do servidor. O contexto enviado não inclui CPF, telefone nem endereço. A pendência é exigir e verificar o token do Firebase em cada pedido, o que também permitiria limitar por usuário. Uma chave de API que foi exposta durante o desenvolvimento deve ser revogada e substituída; chaves nunca devem ficar no aplicativo ou no repositório (Figura 10).

6.8 R7 – Dados calculados ou decididos no aparelho (adulteração)

O preço de um agendamento e a duração são calculados e gravados pelo aplicativo, que um usuário mal-intencionado pode alterar. As regras impedem que o cliente mude o valor depois de criado e que ele conclua o próprio atendimento, mas não verificam se o valor inicial corresponde ao preço do serviço. Um cliente poderia, assim, registrar um agendamento com preço menor, o que distorce o faturamento mostrado ao profissional. O impacto é limitado e a probabilidade é alta para quem sabe usar ferramentas de desenvolvedor.

Risco real. Impacto 3 multiplicado por probabilidade 4 resulta em 12, risco alto. **Pilares afetados:** integridade, irretratabilidade e auditabilidade. **STRIDE:** adulteração

(*Tampering*) e repúdio. **LGPD:** artigo 6º, inciso V (qualidade dos dados) e artigo 46. **Vulnerabilidades associadas:** CWE-602 (aplicação de segurança apenas no lado do cliente); CVSS 3.1, vetor AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:L/A:N, pontuação 4,3.

Situação atual e tratamento. Há limites de tamanho e de formato, e as regras restringem quais campos cada lado pode alterar. A ação prevista é recalcular o valor no servidor (por uma função em nuvem) ou validar nas regras que o valor do agendamento é igual ao do serviço referenciado.

6.9 R8 – Dependências com vulnerabilidades e cadeia de suprimentos

O projeto usa bibliotecas de terceiros nos servidores e carrega os módulos do Firebase de um endereço externo. Vulnerabilidades em qualquer uma delas podem ser exploradas sem que o código do grupo tenha defeito. A auditoria de dependências feita para este relatório apontou 11 vulnerabilidades no serviço de segurança (entre elas, em *nodemailer*, *zod*, *node-forge* e *firebase-admin*) e 2 no assistente (*zod* e *openai*), todas com correção por troca de versão principal (Figura 11). A de maior efeito prático é a do *zod*, que permite consumo excessivo de memória na validação de coleções muito grandes.

Risco real. Impacto 4 multiplicado por probabilidade 3 resulta em 12, risco alto. **Pilares afetados:** disponibilidade, integridade e confidencialidade. **STRIDE:** negação de serviço, adulteração e elevação de privilégio. **LGPD:** artigo 46 e artigo 50 (boas práticas e governança). **Vulnerabilidades associadas:** CWE-1395 (dependência de componente de terceiros vulnerável); CVSS 3.1, vetor AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H, pontuação 7,5.

Situação atual e tratamento. O limite de 8 kilobytes (segurança) e 64 kilobytes (assistente) no corpo das requisições reduz o alcance do problema do *zod*, mas não o elimina, e os serviços são opcionais. As atualizações são de versão principal e exigem teste de regressão, por isso não foram aplicadas sem validação. A ação prevista é atualizar as dependências, rodar a bateria de testes e incluir o *npm audit* como etapa obrigatória de cada entrega.

6.10 R9 – Aparelho perdido, roubado ou emprestado

Quem tem o celular desbloqueado pode abrir o aplicativo e ver os dados do titular. Também é possível ler o armazenamento local do aplicativo em um aparelho com acesso privilegiado. A probabilidade é média, porque perder o celular é comum, e o impacto é médio, limitado à conta do dono do aparelho.

Risco real. Impacto 3 multiplicado por probabilidade 3 resulta em 9, risco médio. **Pilares afetados:** confidencialidade, privacidade e autenticidade. **STRIDE:** divulgação de informação e falsificação de identidade. **LGPD:** artigo 46. **Vulnerabilidades associadas:** CWE-922 (armazenamento inseguro de informação sensível) e CWE-613 (expiração insuficiente de sessão); CVSS 3.1, vetor AV:P/AC:L/PR:N/UI:N/S:U/C:H/I:L/A:N,

pontuação 5,2.

Situação atual e tratamento. A sessão encerra após 15 minutos sem uso, a opção de não manter conectado usa apenas a sessão da janela, o Android não permite cópia de segurança do aplicativo e o armazenamento local guarda apenas a sessão, a última atividade e o cache de CEP. Não há bloqueio por biometria dentro do aplicativo, e o encerramento por inatividade é aplicado no aparelho. A ação prevista é oferecer o desbloqueio por biometria ao reabrir o aplicativo.

6.11 R10 – Indisponibilidade e esgotamento de recursos

O sistema depende do Firebase. Excesso de uso (por ataque ou crescimento) pode esgotar a cota gratuita, e a queda do provedor ou da internet do usuário impede o agendamento. Um outro risco de disponibilidade é a reserva duplicada do mesmo horário, que o desenho evita ao registrar cada horário com um identificador único que só pode ser criado uma vez.

Risco real. Impacto 3 multiplicado por probabilidade 3 resulta em 9, risco médio. **Pilares afetados:** disponibilidade. **STRIDE:** negação de serviço (*Denial of Service*). **LGPD:** artigo 46 e artigo 6º, inciso VII. **Vulnerabilidades associadas:** CWE-400 (consumo descontrolado de recursos); CVSS 3.1, vetor AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:L, pontuação 5,3.

Situação atual e tratamento. O Google absorve a maior parte dos ataques de rede, e os servidores próprios têm limitação de requisições. Faltam monitoramento de cota, alertas de gasto e um plano de continuidade. A ação prevista é configurar alertas de uso e de custo no console e documentar o procedimento de restauração do backup.

6.12 R11 – Governança e conformidade com a LGPD

A lei exige mais do que segurança técnica: base legal para cada tratamento, transparência, canal para o titular, registro das operações e resposta a incidentes. O projeto implementa o consentimento versionado (a versão da política aceita e a data são guardadas e, se a política mudar, o aceite é pedido de novo), a política de privacidade em linguagem simples, a exportação dos dados e a exclusão da conta. As falhas de governança são a ausência de um encarregado designado (o campo de contato da política ainda está sem valor, como mostra a Figura 5), de um procedimento de resposta a incidentes, de um inventário formal das operações de tratamento e de prazos de retenção definidos para os registros de segurança.

Risco real. Impacto 4 multiplicado por probabilidade 3 resulta em 12, risco alto. **Pilares afetados:** legalidade, privacidade e auditabilidade. **STRIDE:** repúdio e divulgação de informação. **LGPD:** artigos 7º e 8º (base legal e consentimento), 9º (acesso facilitado), 18 (direitos do titular), 37 (registro das operações), 41 (encarregado), 48 (comunicação de incidente) e 50 (boas práticas). **Vulnerabilidades associadas:** CWE-359 (exposição de

informação privada a agente não autorizado), como consequência possível; sem pontuação CVSS, por ser um risco de governança.

Situação atual e tratamento. A ação prevista é designar o encarregado e preencher o contato, escrever o plano de resposta a incidentes com prazos de comunicação, montar o registro das operações de tratamento e definir o prazo de guarda dos registros de segurança.

6.13 Ordem de priorização das adequações

A pergunta central deste relatório é qual a ordem de adequações que o BarbTech adotará. A resposta parte da pontuação de risco, em ordem decrescente, e desempata pelo impacto, depois pela exposição à internet e, por fim, pelo custo de correção. O contexto da aplicação pesa em duas decisões: o aplicativo trata dados pessoais e de saúde de clientes comuns, sem equipe de segurança, e tem pouca receita para suportar um incidente, de modo que a conformidade e a proteção da conta valem mais do que o endurecimento de detalhes.

1º, R1 (risco 12). Primeiro vem a autenticação, porque todo o restante depende de uma conta ser difícil de tomar. As ações são a verificação em duas etapas, a exigência de senhas fora de listas de vazamentos e a divulgação do login com Google. **2º, R11 (risco 12).** Em seguida vem a governança: designar o encarregado, preencher o contato da política, escrever o plano de incidentes e o registro das operações, porque o custo é baixo e a exposição legal é imediata. **3º, R8 (risco 12).** Depois, atualizar as dependências e fazer da auditoria uma etapa fixa, porque o problema já está comprovado e a correção é determinística. **4º, R6 (risco 12).** Na sequência, exigir o token do Firebase no assistente e limitar por usuário, o que fecha o único serviço aberto a qualquer pessoa. **5º, R7 (risco 12).** Depois, validar o preço no servidor ou nas regras.

6º, R2 (risco 10). Os riscos de impacto 5 vêm a seguir, mesmo com pontuação menor, porque a probabilidade baixa depende de uma suposição que ainda não foi comprovada: por ser barato, o teste das regras no emulador do Firebase deve ser feito já na primeira etapa, em paralelo às ações anteriores, e a revisão das regras depende dos resultados. **7º, R3 (risco 10).** Restringir o acesso ao console, avaliar criptografia de campo para o CPF e reduzir os dados pedidos. **8º, R4 (risco 10).** Reavaliar o desenho da recuperação com o resultado do segundo fator e oferecer a recuperação por servidor para quem o publicar. **9º, R9 (risco 9).** Desbloqueio por biometria. **10º, R10 (risco 9).** Alertas de cota e plano de restauração. **11º, R5 (risco 8).** Remover scripts embutidos e endurecer a política de conteúdo, deixando por último porque a defesa principal (escape de saída) já está em vigor.

A evolução do nível de maturidade segue o SAMM (OWASP, 2020): o projeto está hoje no nível 1 de governança, projeto, implementação e verificação, com práticas básicas e pontuais; a meta é o nível 2, com práticas repetíveis, que depende de itens como testes automatizados de regras, auditoria na entrega e revisão periódica do modelo de ameaças.

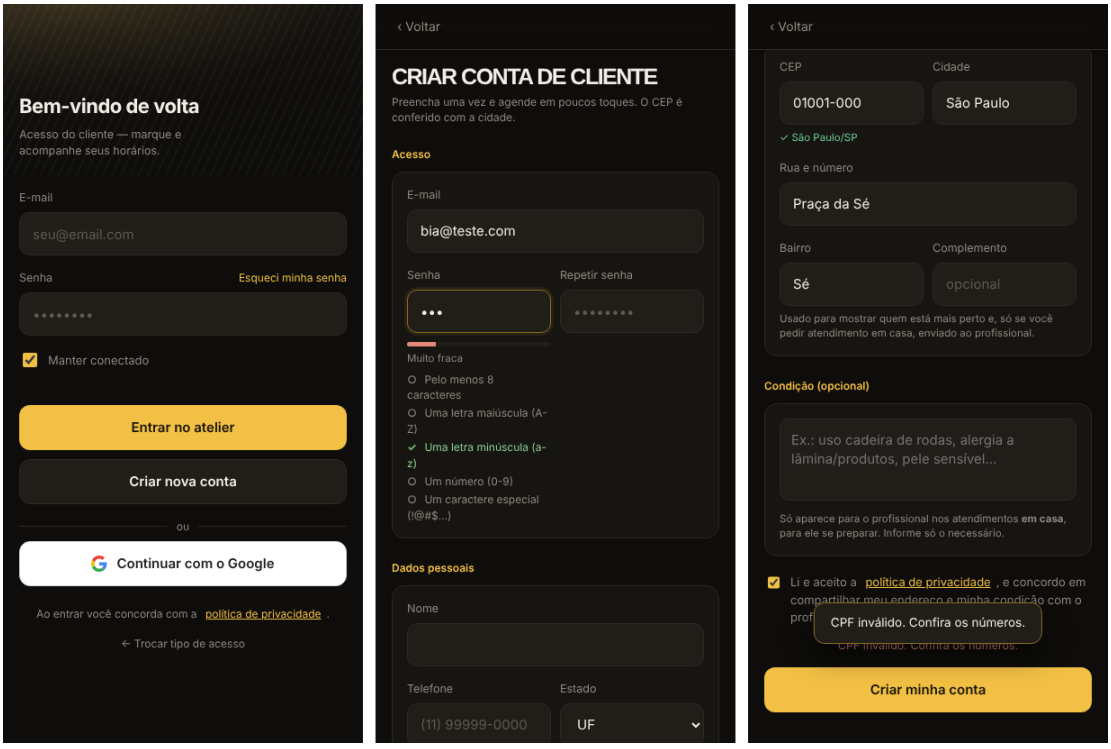
7 VERIFICAÇÃO E EVIDÊNCIAS

Esta seção reúne as evidências visuais. O ambiente de teste é simulado: o Firebase é substituído por um módulo simulado servido ao navegador, o código do aplicativo e dos servidores é o código real, e os comandos foram executados em um terminal de Linux. Não houve teste com o Firebase real, com envio real de e-mail, com o aplicativo em um celular físico, nem com as regras no emulador do Firebase. As ferramentas usadas foram o draw.io (diagramas), o Playwright (navegador automatizado e capturas das telas), o executor de testes do Node, o *curl*, o *grep* e o *npm audit*.

7.1 Telas do aplicativo

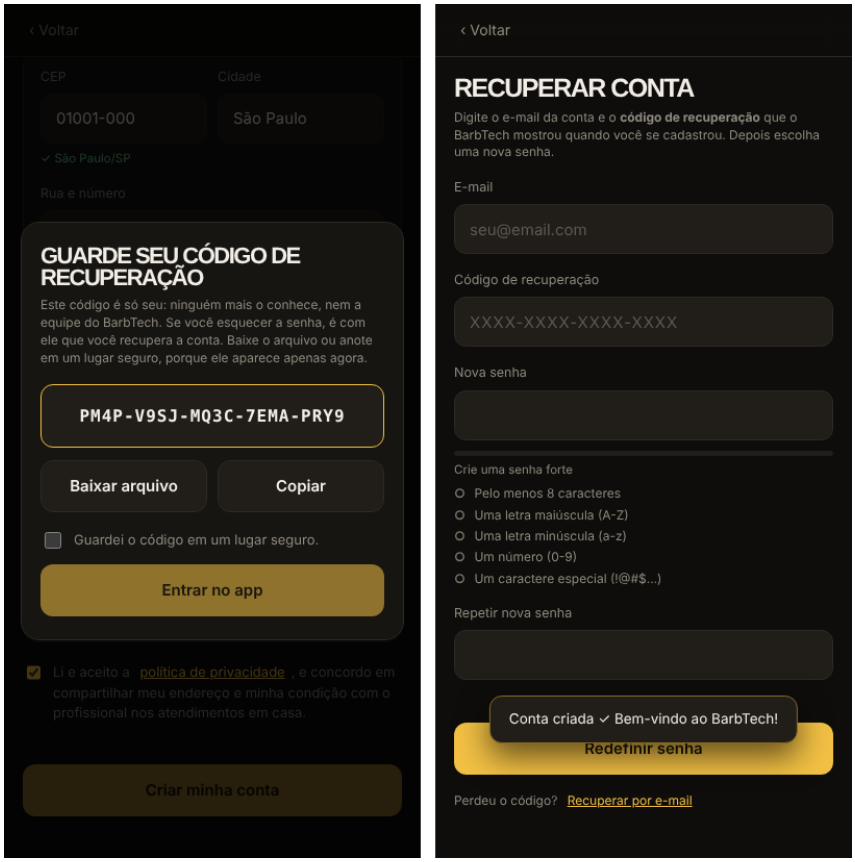
A Figura 3 mostra a tela de login com a opção de login com Google, a lista de requisitos de senha ainda não atendidos e a recusa de um CPF cujos dígitos verificadores não conferem. A Figura 4 mostra a janela exibida no cadastro, com o código de recuperação, os botões de baixar e copiar e a confirmação obrigatória, e a tela de recuperação de conta. A Figura 5 mostra a tela de segurança e privacidade, com o código de recuperação, os consentimentos e os direitos do titular, e o início da política de privacidade.

Figura 3 – Login, requisitos de senha e recusa de CPF inválido



Fonte: elaborado pelo autor (2026).

Figura 4 – Código de recuperação exibido no cadastro e tela de recuperação de conta



Fonte: elaborado pelo autor (2026).

Figura 5 – Segurança e privacidade e política de privacidade

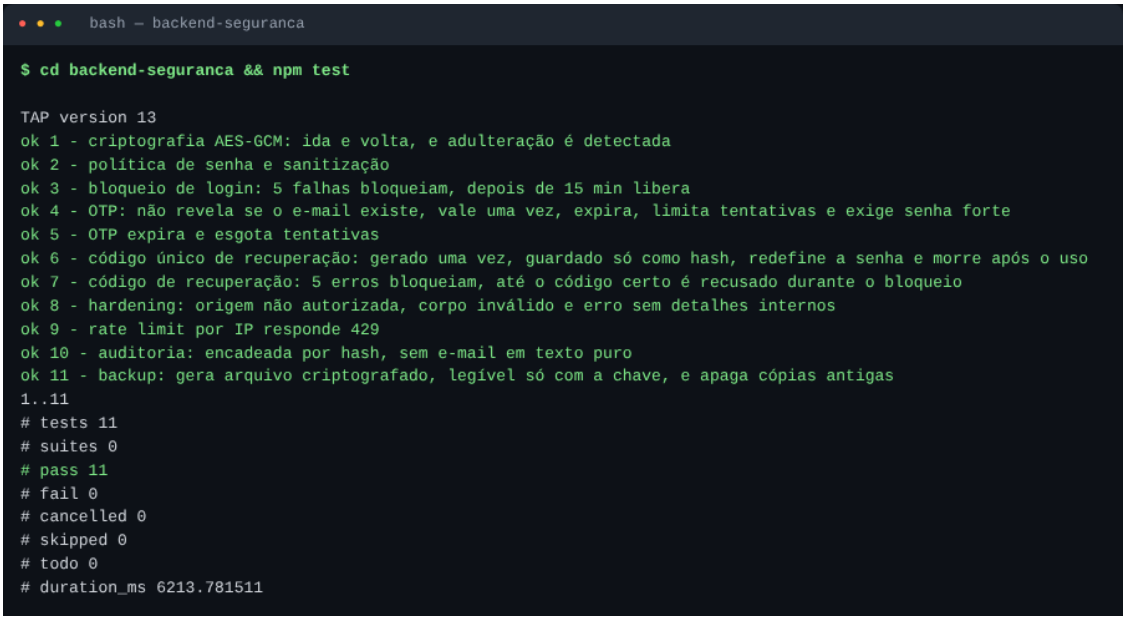


Fonte: elaborado pelo autor (2026).

7.2 Testes automatizados e comandos

A Figura 6 mostra a execução dos 11 testes automatizados do serviço de segurança, que cobrem criptografia, política de senha, bloqueio de login, código por e-mail, origem não autorizada, limitação de requisições, auditoria e backup. Os testes passaram. Observação: o teste de número 6 se refere ao mecanismo de código de recuperação do servidor, que continua disponível, mas não é mais o que o aplicativo usa.

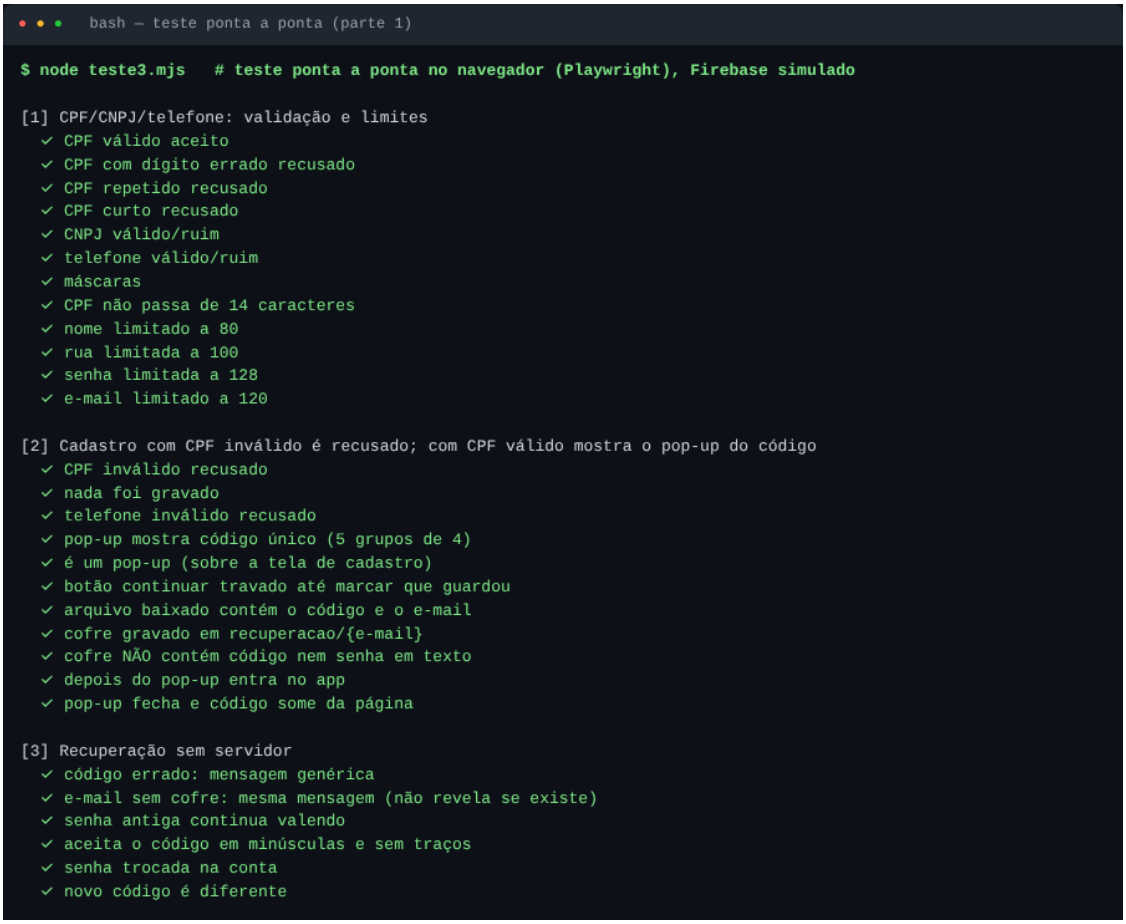
Figura 6 – Execução dos testes automatizados do serviço de segurança



Fonte: elaborado pelo autor (2026).

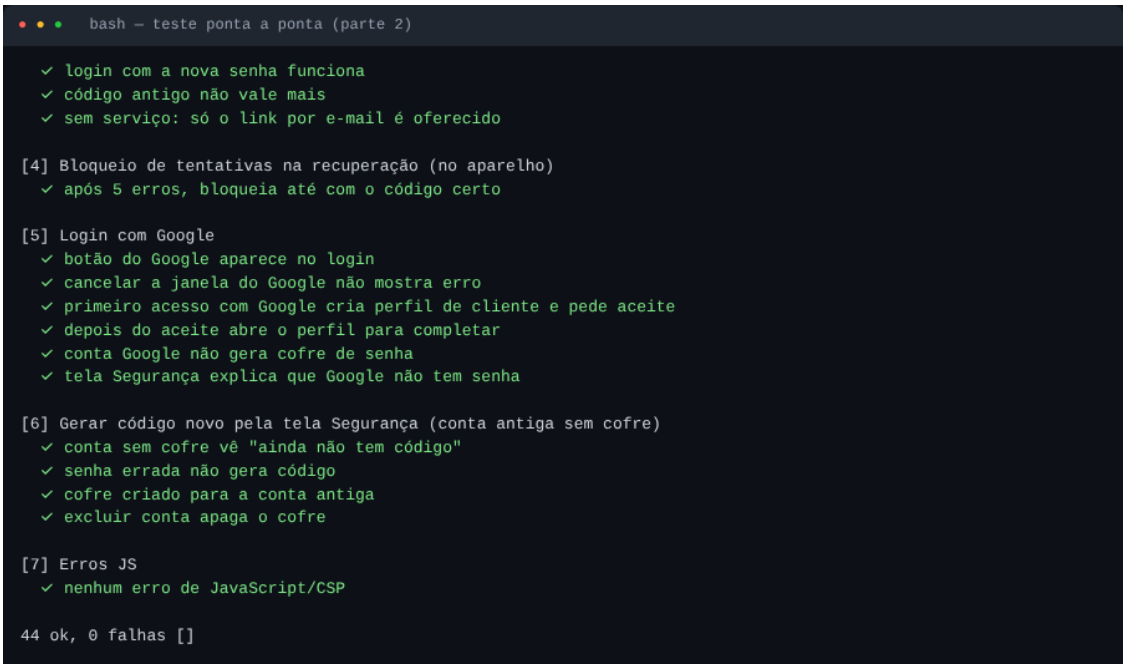
As Figuras 7 e 8 mostram o teste ponta a ponta executado em navegador automatizado sobre o código real do aplicativo: validação de CPF, CNPJ e telefone e limites de tamanho, criação de conta com a janela do código, o arquivo baixado, a confirmação de que o cofre não contém a senha nem o código, a recuperação com o código (inclusive em letras minúsculas e sem traços), a recusa do código antigo, o bloqueio após cinco tentativas, o login com Google (inclusive o cancelamento da janela), a geração de código pela tela de segurança e a exclusão da conta com a remoção do cofre. Foram 44 verificações, todas aprovadas.

Figura 7 – Teste ponta a ponta, parte 1: validação, cadastro e janela do código



Fonte: elaborado pelo autor (2026).

Figura 8 – Teste ponta a ponta, parte 2: recuperação, bloqueio, Google e exclusão



Fonte: elaborado pelo autor (2026).

A Figura 9 mostra o servidor de segurança em execução com o módulo do Firebase simulado. A primeira consulta exibe os cabeçalhos de segurança devolvidos. A segunda mostra a recusa (403) de um pedido vindo de uma origem fora da lista. A terceira mostra a resposta genérica a uma entrada inválida. A última dispara 130 requisições seguidas e conta os códigos de resposta: 118 aceitas e 12 recusadas com 429, o que demonstra a limitação de requisições por IP.

Figura 9 – Cabeçalhos de segurança, recusa de origem não autorizada, validação e limitação de requisições

```
bash — servidor de segurança em ambiente simulado

$ curl -i http://localhost:3199/health
HTTP/1.1 200 OK
Content-Security-Policy: default-src 'self';base-uri 'self';font-src 'self' https: data:;form-action 'self';fra
me-ancestors 'self';img-src 'self' data:;object-src 'none';script-src 'self';script-src-attr 'none';style-src
'self' https: 'unsafe-inline';upgrade-insecure-requests
Referrer-Policy: no-referrer
Strict-Transport-Security: max-age=15552000; includeSubDomains
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Vary: Origin
Content-Type: application/json; charset=utf-8
Content-Length: 11

{"ok":true}

$ curl -s -X POST http://localhost:3199/api/seg/pode-entrar -H "Origin: https://site-malicioso.example" -d "{}"
-H "Content-Type: application/json" -i | head -4
HTTP/1.1 403 Forbidden
Content-Security-Policy: default-src 'self';base-uri 'self';font-src 'self' https: data:;form-action 'self';fra
me-ancestors 'self';img-src 'self' data:;object-src 'none';script-src 'self';script-src-attr 'none';style-src
'self' https: 'unsafe-inline';upgrade-insecure-requests
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Resource-Policy: same-origin

$ curl -s -X POST .../api/seg/pode-entrar -d '{"email":"nao-e-email"}'   # entrada inválida
{"erro":"Dados inválidos."}

$ for i in $(seq 1 130); do curl -s -o /dev/null -w "%{http_code}\n" http://localhost:3199/health; done | sort
| uniq -c
    118 200
     12 429
```

Fonte: elaborado pelo autor (2026).

A Figura 10 mostra a varredura de segredos no código (chaves de API da OpenAI, chaves de API do Google e chaves privadas), que não encontrou ocorrências (código de saída 1 do *grep*, que significa nenhuma linha encontrada). Os dados de configuração do Firebase no aplicativo não são segredos, mas devem ser preenchidos com os do próprio projeto. A Figura 11 mostra o resultado da auditoria de dependências, que é o achado negativo mais concreto desta verificação, tratado no risco R8.

Figura 10 – Varredura de segredos no repositório

```
bash — varredura de segredos no código

$ grep -rnE "sk-[A-Za-z0-9]{20,}|AIza[0-9A-Za-z_-]{35}|-----BEGIN [A-Z ]*PRIVATE KEY" --include=*.js --include=
*.html --include=*.ts --include=*.json --include=*.md --exclude-dir=node_modules .
(nenhuma ocorrência)
$ echo $?
1
```

Fonte: elaborado pelo autor (2026).

Figura 11 – Auditoria de dependências dos dois servidores Node



Fonte: elaborado pelo autor (2026).

8 CONSIDERAÇÕES FINAIS

O BarbTech foi projetado com a segurança presente desde a arquitetura: autenticação delegada a um provedor especializado, autorização por regras avaliadas no servidor do banco, dados separados por sensibilidade, mínimo necessário nos serviços de terceiros, consentimento versionado, direitos do titular implementados e opção de login com Google. A análise de onze riscos mostra que nenhum deles é crítico segundo a escala adotada, mas cinco são altos por causa de lacunas concretas: ausência de segundo fator, governança da LGPD ainda sem encarregado, dependências com vulnerabilidades, assistente aberto a qualquer pessoa e preço calculado no aparelho.

As principais limitações do trabalho são três. A primeira é que os testes usaram o Firebase simulado e, portanto, não comprovam o comportamento das regras de segurança reais nem do envio de e-mail e de avisos. A segunda é que várias proteções (política de senha, bloqueio de tentativas, encerramento da sessão por inatividade, limites de tamanho dos campos) são aplicadas no aparelho, que pode ser alterado, e só são garantidas do lado do servidor onde isso está dito expressamente. A terceira é que as pontuações de risco e de CVSS são estimativas do grupo e devem ser revistas com dados reais de uso. Os próximos passos, na ordem da seção 6.13, são atualizar dependências, exigir login no assistente, ativar a verificação em duas etapas, executar as regras no emulador e formalizar a governança da LGPD.

REFERÊNCIAS

BRASIL. **Lei nº 13.709, de 14 de agosto de 2018**. Lei Geral de Proteção de Dados Pessoais (LGPD). Brasília, DF: Presidência da República, 2018.

FIRST. **Common Vulnerability Scoring System v3.1: specification document**. [S. l.]: Forum of Incident Response and Security Teams, 2019. Disponível em: https://www.first.org/cvss/v3.1/specification-document. Acesso em: __ out. 2026.

GOOGLE CLOUD. **Default encryption at rest**. [S. l.]: Google, 2024. Disponível em: https://cloud.google.com/docs/security/encryption/default-encryption. Acesso em: __ out. 2026.

IETF. **RFC 6749: the OAuth 2.0 authorization framework**. [S. l.]: Internet Engineering Task Force, 2012. Disponível em: https://www.rfc-editor.org/rfc/rfc6749. Acesso em: __ out. 2026.

IETF. **RFC 7519: JSON Web Token (JWT)**. [S. l.]: Internet Engineering Task Force, 2015. Disponível em: https://www.rfc-editor.org/rfc/rfc7519. Acesso em: __ out. 2026.

MITRE. **Common Weakness Enumeration (CWE)**. [S. l.]: The MITRE Corporation, 2024. Disponível em: https://cwe.mitre.org/. Acesso em: __ out. 2026.

NIST. **Cybersecurity Framework (CSF) 2.0**. Gaithersburg: National Institute of Standards and Technology, 2024. Disponível em: https://www.nist.gov/cyberframework. Acesso em: __ out. 2026.

NIST. **SP 800-63B: digital identity guidelines: authentication and lifecycle management**. Gaithersburg: National Institute of Standards and Technology, 2017. Disponível em: https://pages.nist.gov/800-63-3/sp800-63b.html. Acesso em: __ out. 2026.

OWASP FOUNDATION. **OWASP Top 10:2021**. [S. l.]: OWASP Foundation, 2021a. Disponível em: https://owasp.org/Top10/. Acesso em: __ out. 2026.

OWASP FOUNDATION. **OWASP Application Security Verification Standard (ASVS) 4.0.3**. [S. l.]: OWASP Foundation, 2021b. Disponível em: https://owasp.org/www-project-application-security-verification-standard/. Acesso em: __ out. 2026.

OWASP FOUNDATION. **OWASP Software Assurance Maturity Model (SAMM) v2**. [S. l.]: OWASP Foundation, 2020. Disponível em: https://owaspsamm.org/. Acesso em: __ out. 2026.

OWASP FOUNDATION. **OWASP Mobile Application Security Verification Standard (MASVS)**. [S. l.]: OWASP Foundation, 2023. Disponível em: https://mas.owasp.org/MASVS/. Acesso em: __ out. 2026.

SHOSTACK, Adam. **Threat modeling: designing for security**. Indianapolis: John Wiley & Sons, 2014.